

Documentacion Tecnica

# repo\_analysis

Generado el 04 de june de 2026

11 archivos documentados

---

Generado con Ollama llama3.2 - Axet Plugin

## Indice de contenidos

---

- [backend/app.py](#) (Python)
- [backend/database.py](#) (Python)
- [backend/models.py](#) (Python)
- [database/schema.sql](#) (SQL)
- [frontend/app.js](#) (JavaScript)
- [frontend/index.html](#) (HTML)
- [backend/ml\\_service.py](#) (Python)
- [ml/predict.py](#) (Python)
- [ml/train\\_model.py](#) (Python)
- [.docgen\\_checkpoint.json](#) (JSON)
- [frontend/src/components/test.txt/MiComponente.js](#) (JavaScript)

# Resumen ejecutivo del proyecto

## Descripción del Proyecto

El proyecto `repo_analysis` es una aplicación web que analiza y predice las ventas de arepas basadas en datos climáticos. Utiliza tecnologías como FastAPI, SQLAlchemy y ML para gestionar inventarios y realizar predicciones.

## Tecnologías utilizadas

- Lenguajes: Python
- Frameworks: FastAPI, Flask
- Librerías: SQLAlchemy, NumPy, Pandas

## Arquitectura

La arquitectura del proyecto se compone de varias capas:

- **Capa de presentación:** La capa frontend, construida con HTML, CSS y JavaScript, proporciona una interfaz para los usuarios.
- **Capa de negocio:** La capa backend, construida con FastAPI y SQLAlchemy, gestiona la lógica de negocio y la base de datos.
- **Capa de inteligencia artificial:** La capa ML, construida con Python y bibliotecas como scikit-learn, se encarga de entrenar modelos y realizar predicciones.

## Módulos principales

- `backend/app.py`: Define la estructura básica de una aplicación web FastAPI para gestionar inventarios y ventas de arepas.
- `database/schema.sql`: Define la estructura de una base de datos para la gestión de inventarios y ventas de arepas.
- `frontend/app.js`: Es el punto de entrada principal del frontend de una aplicación que realiza predicciones climáticas utilizando un modelo entrenado previamente.
- `ml/predict.py`: Es el punto de entrada principal para la predicción de ventas utilizando un modelo entrenado con datos previamente almacenados.

## Flujo general del sistema

El flujo general del sistema es el siguiente:

1. Un usuario ingresa los datos necesarios en la interfaz web.
2. La capa frontend envía los datos a la capa backend, que los procesa y los almacena en la base de datos.
3. La capa ML utiliza un modelo entrenado para realizar predicciones sobre las ventas basadas en los datos climáticos.
4. La capa backend envía las predicciones al frontend, que las muestra al usuario.

## Arbol de archivos del proyecto

```
repo_analysis/  
  .docgen_checkpoint.json  
  backend/  
    app.py  
    database.py  
    ml_service.py  
    models.py
```

```
■■■ database/
■   ■■■ schema.sql
■■■ frontend/
■   ■■■ app.js
■   ■■■ index.html
■   ■■■ src/
■   ■   ■■■ components/
■   ■   ■   ■■■ test.txt/
■   ■   ■   ■   ■■■ MiComponente.js
■■■ ml/
■   ■■■ predict.py
■   ■■■ train_model.py
```

backend/app.py

Python

### Propósito

Este archivo define la estructura básica de una aplicación web FastAPI para gestionar inventarios y ventas de arepas, utilizando tecnologías como SQLAlchemy y Pydantic.

### Componentes principales

| Nombre    | Tipo     | Descripción                                       |
|-----------|----------|---------------------------------------------------|
| app       | Clase    | Instancia principal de la aplicación web.         |
| Base      | Clase    | Clase base para la base de datos.                 |
| engine    | Variable | Conexión a la base de datos.                      |
| ■■ get_db | Función  | Función que devuelve una sesión de base de datos. |

### Ejemplos de uso

```
# Ejemplo 1: muestra cómo llamar al componente principal o función más importante
from app import app

# Inicializa la aplicación web
if __name__ == "__main__":
    app.run()

# Ejemplo 2 (opcional): caso de uso secundario o manejo de error
# No hay casos de uso secundarios relevantes en este archivo.
```

### Dependencias

- fastapi: biblioteca para crear aplicaciones web rápidas y escalables.
- pydantic: biblioteca para validación y serialización de datos.
- sqlalchemy: biblioteca para interactuar con bases de datos.
- corsmiddleware: middleware para manejar solicitudes CORS.

### Flujo de datos

La aplicación recibe solicitudes HTTP y procesa las ventas de arepas, almacenando los datos en la base de datos. La respuesta se envía al cliente.

### Notas técnicas

- Se utiliza SQLAlchemy para interactuar con la base de datos.
- Se utiliza Pydantic para validar y serializar los datos.
- El middleware CORSMiddleware permite que las solicitudes CORS sean tratadas correctamente.

### ■■ Advertencias de validación

Los siguientes componentes fueron mencionados por el LLM pero **no se encontraron verbatim en el código fuente**. Pueden ser alucinaciones o alias:

- getdb

backend/database.py

Python

Propósito

Este archivo de código define la conexión a una base de datos utilizando SQLAlchemy y proporciona un mecanismo para obtener una sesión de la base de datos.

Componentes principales

| Nombre   | Tipo    | Descripción                             |
|----------|---------|-----------------------------------------|
| ■ get_db | Función | Regresa una sesión de la base de datos. |

Ejemplos de uso

```
# Ejemplo 1: muestra cómo llamar al componente principal o función más importante
db = await get_db()

# Ejemplo 2 (opcional): caso de uso secundario o manejo de error
try:
    db = await get_db()
except Exception as e:
    print(f"Error al obtener la sesión de la base de datos: {e}")
```

Dependencias

- sqlalchemy
- fastapi

Flujo de datos

El archivo utiliza una función `get_db` que recibe como entrada un objeto `Depends` y devuelve una sesión de la base de datos. La sesión se utiliza para interactuar con la base de datos.

Notas técnicas

- Se utiliza el patrón de conexión "check\_same\_thread" para evitar problemas de seguridad.
- El archivo utiliza la función `SessionLocal` para obtener una sesión de la base de datos, que es un objeto que se puede utilizar para realizar operaciones en la base de datos.

■ Advertencias de validación

Los siguientes componentes fueron mencionados por el LLM pero **no se encontraron verbatim en el código fuente**. Pueden ser alucinaciones o alias:

- getdb

backend/models.py

Python

**Propósito**  
Este archivo define las clases de modelos para la base de datos del sistema, que incluyen Inventario, Venta y VariableExterna.

Componentes principales

| Nombre          | Tipo  | Descripción                                         |
|-----------------|-------|-----------------------------------------------------|
| Inventario      | Clase | Modelo de datos para el inventario de ingredientes. |
| Venta           | Clase | Modelo de datos para las ventas.                    |
| VariableExterna | Clase | Modelo de datos para variables externas.            |

Ejemplos de uso

```
# Ejemplo 1: muestra cómo llamar al componente principal o función más importante
inventario = Inventario(ingrediente="Arroz", cantidad_actual=100.00, unidad="Kg")
venta = Venta(fecha="2023-03-01", tipo_arepa="Arepa de queso", cantidad=2, precio_unitario=10.99)

# Ejemplo 2 (opcional): caso de uso secundario o manejo de error
try:
    venta = Venta(fecha="2023-03-01", tipo_arepa="Arepa de queso", cantidad=-1, precio_unitario=10.99)
except ValueError as e:
    print(e) # Manejo de error: cantidad no puede ser negativa
```

**Dependencias**

- sqlalchemy
- .database (módulo base)

**Flujo de datos**  
El flujo de datos es el siguiente: se recibe una solicitud para crear o actualizar un registro en la base de datos, se procesa la solicitud y se retorna el resultado.

- Notas técnicas**
- Se utiliza SQLAlchemy como ORM para interactuar con la base de datos.
  - Las clases de modelo están definidas utilizando la sintaxis de SQLAlchemy.
  - Los campos de cada clase corresponden a las columnas de la tabla en la base de datos.

database/schema.sql

SQL

**Propósito**  
Este archivo de código define la estructura de una base de datos para la gestión de inventarios y ventas de arepas.

## Componentes principales

| Nombre                             | Tipo  | Descripción                                                                                                        |
|------------------------------------|-------|--------------------------------------------------------------------------------------------------------------------|
| <code>inventario</code>            | Tabla | Almacena información sobre los componentes del inventario, como el ingrediente, cantidad actual y unidad.          |
| <code>ventas</code>                | Tabla | Registra las ventas de arepas, incluyendo la fecha, tipo de arepa, cantidad vendida y precio unitario.             |
| ■■ <code>variables_externas</code> | Tabla | Almacena información sobre variables externas que pueden afectar el negocio, como el clima y si es un día festivo. |

## Ejemplos de uso

```
-- Muestra cómo crear un nuevo inventario
CREATE TABLE IF NOT EXISTS inventario (
  id SERIAL PRIMARY KEY,
  ingrediente VARCHAR(50) NOT NULL,
  cantidad_actual NUMERIC(10, 2) NOT NULL DEFAULT 0,
  unidad VARCHAR(20) NOT NULL
);

-- Muestra cómo registrar una venta
INSERT INTO ventas (fecha, tipo_arepa, cantidad, precio_unitario)
VALUES ('2023-03-01', 'Arepa de queso', 10, 5.99);

-- Caso de uso secundario: mostrar información sobre un inventario específico
SELECT * FROM inventario WHERE id = 1;
```

## Dependencias

Ninguna

## Flujo de datos

El flujo de datos es el siguiente:

- Se recibe la fecha y la cantidad vendida.
- Se procesa la venta y se actualiza el inventario correspondiente.
- Se retorna la información sobre la venta.

## Notas técnicas

- El uso de `SERIAL` como tipo de clave primaria garantiza que las claves primarias sean autoincrementables.
- La utilización de `NUMERIC(10, 2)` para almacenar números decimales permite una precisión de dos decimales.

## ■■ Advertencias de validación

Los siguientes componentes fueron mencionados por el LLM pero **no se encontraron verbatim en el código fuente**. Pueden ser alucinaciones o alias:

- `variablesexternas`



frontend/app.js

JavaScript

**Propósito**

Este archivo de código (`app.js`) es el punto de entrada principal del frontend de una aplicación que realiza predicciones climáticas utilizando un backend FastAPI.

**Componentes principales**

| Nombre                               | Tipo    | Descripción                                                     |
|--------------------------------------|---------|-----------------------------------------------------------------|
| <code>fetch</code>                   | Función | Utilizada para realizar solicitudes HTTP a la API del backend.  |
| <code>document.getElementById</code> | Método  | Se utiliza para acceder a elementos HTML y obtener sus valores. |

**Ejemplos de uso**

```
// Llama al componente principal para enviar una solicitud POST a la API del backend.
const form = document.getElementById("prediccion-form");
form.addEventListener("submit", async (e) => {
  // ...
});

// Obtiene el valor de un elemento HTML y lo utiliza en la solicitud.
const fecha = document.getElementById("fecha").value;

# Ejemplo 2: manejo de error
try {
  const res = await fetch(`${BASE_URL}/prediccion`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(body)
  });

  if (!res.ok) {
    throw new Error(`Error ${res.status}: ${res.statusText}`);
  }

  const data = await res.json();
} catch (error) {
  console.error(error);
}
```

**Dependencias**

- `fetch`
- `JSON`
- `application/json` (en la cabecera de solicitud)

**Flujo de datos**

El flujo de datos es el siguiente:

- Se recibe como entrada: los valores del formulario (`fecha`, `clima`, `es_festivo`)
- Se procesa: se crea un objeto con los valores obtenidos y se envía a la API del backend mediante una solicitud POST.
- Se retorna/produce: la respuesta de la API del backend, que se convierte en el resultado de la predicción.

## Notas técnicas

- La URL del backend (`BASE_URL`) es dinámica y depende de dónde corra FastAPI.
- Se utiliza `JSON.stringify` para convertir el objeto a un string JSON antes de enviarlo en la solicitud.

frontend/index.html

HTML

## Propósito

Este archivo de código define la estructura básica de una página web que permite a los usuarios ingresar datos y obtener predicciones sobre la producción.

## Componentes principales

| Nombre                                           | Tipo     | Descripción                                                     |
|--------------------------------------------------|----------|-----------------------------------------------------------------|
| <code>fecha</code>                               | Input    | Campo de texto para ingresar la fecha                           |
| <code>clima</code>                               | Input    | Campo de texto para seleccionar el clima (templado, frío, etc.) |
| <code><input type="checkbox"/> es_festivo</code> | Checkbox | Opción para indicar si es un día festivo                        |
| <code>prediccion-form</code>                     | Form     | Formulario que envía los datos al servidor                      |

## Ejemplos de uso

```
# Ejemplo 1: muestra cómo llamar al componente principal o función más importante
document.getElementById('prediccion-form').submit();

# Ejemplo 2 (opcional): caso de uso secundario o manejo de error
if (!document.getElementById('fecha').value) {
    alert("Por favor, ingresa una fecha");
}
```

## Dependencias

- `app.js` (archivo JavaScript que contiene la lógica de negocio)

## Flujo de datos

El formulario recibe los datos ingresados por el usuario y envía un evento al servidor para obtener la predicción. El resultado se muestra en el elemento .

## Notas técnicas

- Se utiliza el atributo `required` para que el campo de fecha sea obligatorio.
- Se utiliza el atributo `value` para establecer un valor predeterminado para el campo de clima.
- Es importante verificar si el campo de fecha está vacío antes de enviar la solicitud al servidor.

## ■ ■ Advertencias de validación

Los siguientes componentes fueron mencionados por el LLM pero **no se encontraron verbatim en el código fuente**. Pueden ser alucinaciones o alias:

- esfestivo

backend/ml\_service.py

Python

Propósito

Este archivo de código contiene la lógica de negocio para un servicio de aprendizaje automático (ML) que utiliza un modelo entrenado previamente. El módulo permite cargar el modelo y procesar datos de entrada para generar features relevantes.

Componentes principales

| Nombre                | Tipo    | Descripción                                                             |
|-----------------------|---------|-------------------------------------------------------------------------|
| ■■ cargar_modelo      | Función | Carga el modelo entrenado desde archivo joblib.                         |
| ■■ construir_features | Función | Genera features a partir de una fecha, clima y indicador de festividad. |

Ejemplos de uso

```
# Ejemplo 1: muestra cómo llamar al componente principal o función más importante
fecha = date(2022, 12, 25)
clima = "Cálido"
es_festivo = True
model = cargar_modelo()
features = construir_features(fecha, clima, es_festivo)
print(features)

# Ejemplo 2 (opcional): caso de uso secundario o manejo de error
# No hay casos de uso secundarios relevantes en este código.
```

Dependencias

- pandas
- joblib
- datetime

Flujo de datos

El módulo procesa una fecha, clima y un indicador de festividad como entrada. Genera features a partir de estos datos y las retorna.

Notas técnicas

- El modelo se carga desde archivo joblib para facilitar la actualización o reemplazo del modelo.
- La función `construir_features` utiliza una codificación simple para generar features, lo que podría ser mejorado con un enfoque más robusto.

## ■ ■ Advertencias de validación

Los siguientes componentes fueron mencionados por el LLM pero **no se encontraron verbatim en el código fuente**. Pueden ser alucinaciones o alias:

- cargarmodelo
- construirfeatures

ml/predict.py

Python

## Propósito

Este archivo de código es el punto de entrada principal para la predicción de ventas utilizando un modelo entrenado con datos previamente almacenados en un archivo de joblib.

## Componentes principales

| Nombre | Tipo     | Descripción                                                          |
|--------|----------|----------------------------------------------------------------------|
| main   | Función  | Es la función principal que carga el modelo y realiza la predicción. |
| model  | Variable | Almacena el modelo entrenado cargado desde el archivo de joblib.     |

## Ejemplos de uso

```
# Ejemplo 1: muestra cómo llamar al componente principal o función más importante
import ml.predict as predict

predict.main()

# Ejemplo 2 (opcional): caso de uso secundario o manejo de error
# No hay casos de uso secundarios relevantes en este archivo.
```

## Dependencias

- joblib
- pandas

## Flujo de datos

El flujo de datos es el siguiente: se recibe un conjunto de datos de prueba, se carga el modelo entrenado y se realiza la predicción utilizando ese modelo.

## Notas técnicas

- El archivo utiliza joblib para cargar y guardar modelos.
- La función `main` es la entrada principal del script y carga el modelo antes de realizar la predicción.

ml/train\_model.py

Python

## Propósito

Este archivo de código es responsable de entrenar un modelo lineal de regresión utilizando datos de ventas totales y características como temperatura, si es festivo y día de la semana.

## Componentes principales

| Nombre                        | Tipo    | Descripción                                                                 |
|-------------------------------|---------|-----------------------------------------------------------------------------|
| <code>main</code>             | Función | La función principal que ejecuta el entrenamiento del modelo y su guardado. |
| <code>LinearRegression</code> | Clase   | La clase utilizada para entrenar el modelo lineal de regresión.             |

## Ejemplos de uso

```
# Ejemplo 1: muestra cómo llamar al componente principal o función más importante
df = pd.read_csv(DATASET_PATH)
X = df[["temperatura", "es_festivo", "dia_semana"]]
y = df["ventas_totales"]
model = LinearRegression()
model.fit(X, y)
joblib.dump(model, MODEL_PATH)
```

```
# Ejemplo 2 (opcional): caso de uso secundario o manejo de error
# No hay casos de uso secundarios relevantes en este archivo.
```

## Dependencias

- `pandas` como biblioteca para manipular datos
- `sklearn.linear_model` como biblioteca para modelos lineales de regresión
- `joblib` como biblioteca para serializar y deserializar objetos
- `os` como biblioteca para manejar sistemas de archivos

## Flujo de datos

El flujo de datos es el siguiente:

- Se lee un archivo CSV llamado "dataset\_example.csv" desde la variable de entorno `DATASET_PATH`.
- Se extraen las columnas "temperatura", "es\_festivo" y "dia\_semana" como características (X) y se asigna al valor de las ventas totales ("ventas\_totales") a la variable y.
- Se crea un objeto `LinearRegression` y se le pasa las características X y el valor objetivo y para entrenar el modelo.
- El modelo se guarda en un archivo llamado "model.joblib" utilizando la biblioteca `joblib`.

## Notas técnicas

- Es importante mencionar que este código utiliza variables de entorno para almacenar los caminos a los datos y al modelo, lo cual es una buena práctica para mantener la flexibilidad y reutilizar el código.
- La función `main` es la entrada principal del programa y se encarga de ejecutar el entrenamiento del modelo y su guardado.

.docgen\_checkpoint.json

JSON

### Propósito

Este archivo define la estructura básica de una aplicación web FastAPI para gestionar inventarios y ventas de arepas, utilizando tecnologías como SQLAlchemy y Pydantic.

### Componentes principales

| Nombre | Tipo     | Descripción                               |
|--------|----------|-------------------------------------------|
| app    | Clase    | Instancia principal de la aplicación web. |
| Base   | Clase    | Clase base para la base de datos.         |
| engine | Variable | Conexión a la base de datos.              |

### Ejemplos de uso

```
# Ejemplo 1: muestra cómo llamar al componente principal o función más importante
from app import app
app.listen(8000)
print("Servidor escuchando en el puerto 8000")

# Ejemplo 2 (opcional): caso de uso secundario o manejo de error
import logging
try:
    # Código que puede fallar
except Exception as e:
    logging.error(f"Error: {e}")
```

### Dependencias

- FastAPI
- SQLAlchemy
- Pydantic
- logging (para manejo de errores)

### Flujo de datos

El flujo de datos se centra en la gestión de inventarios y ventas de arepas. La aplicación recibe como entrada los datos de los inventarios y las ventas, los procesa utilizando SQLAlchemy y Pydantic, y retorna los resultados.

### Notas técnicas

- Se utiliza FastAPI para crear una aplicación web rápida y escalable.
- SQLAlchemy se utiliza para interactuar con la base de datos.
- Pydantic se utiliza para validar los datos de entrada y salida.
- El manejo de errores se realiza utilizando el módulo logging.

frontend/src/components/test.txt/MiComponente.js

JavaScript

## Propósito

Este archivo contiene la definición de un componente JavaScript llamado **MiComponente**, que es una función que devuelve un elemento HTML.

## Componentes principales

| Nombre       | Tipo    | Descripción                                         |
|--------------|---------|-----------------------------------------------------|
| MiComponente | Función | Un componente que muestra un mensaje de bienvenida. |

## Ejemplos de uso

```
import { MiComponente } from './MiComponente';

const App = () => {
  return (
    <div>
      <MiComponente />
    </div>
  );
};

# Ejemplo 2: caso de uso secundario (opcional)
import { MiComponente } from './MiComponente';
import React from 'react';

const ErrorBoundary = () => {
  try {
    return <MiComponente />;
  } catch (error) {
    console.error(error);
    return null;
  }
};
```

## Dependencias

Ninguna

## Flujo de datos

El componente **MiComponente** recibe como entrada none y procesa el mensaje de bienvenida. Retorna un elemento HTML que contiene el mensaje.

## Notas técnicas

No hay patrones de diseño específicos usados en este componente, pero se recomienda mejorar la reutilización del código mediante la creación de una función más generica para mostrar mensajes de bienvenida.